

Patrones útiles en Go

Por: Ricardo Cuellar



Patrones útiles en Go

- Patrón repository
- Patrón strategy
- Patrón adapter
- Patrón middleware



Patrón repository

Patrón repository: Separar lógica de negocio del acceso de datos

```
example.go

type TaskRepository interface {
    FindAll() ([]Task, error)
    Create(task Task) error
}

type TaskService struct {
    repo TaskRepository
}

func NewTaskService(repo TaskRepository) *TaskService {
    return &TaskService{repo: repo}
}
```



Patrón factory

Patrón factory: Centralizar la creación de objetos cuando construirlos empieza a ser repetitivo o muy complejo.

```
example.go
func NewTaskService(repo TaskRepository, logger *slog.Logger) *TaskService {
    return &TaskService{
        repo:    repo,
        logger:  logger,
    }
}
```



Patrón strategy

Patrón strategy: Permite cambiar un comportamiento sin modificar el código principal.

```
example.go

type PaymentProcessor interface {
    Pay(amount float64) error
}

type StripeProcessor struct{}

func (s StripeProcessor) Pay(amount float64) error {
    return nil
}

type PayPalProcessor struct{}

func (p PayPalProcessor) Pay(amount float64) error {
    return nil
}
```

```
example.go

type CheckoutService struct {
    processor PaymentProcessor
}
```



Patrón adapter

Patrón adapter: Permite conectar tu aplicación con una librería, API externa o sistema que tiene una interfaz distinta a la que necesitamos.

```
example.go

type EmailSender interface {
    Send(to string, subject string, body string) error
}
```

```
example.go

type ExternalMailClient struct{}

func (c ExternalMailClient) SendEmail(payload MailPayload) error {
    return nil
}
```

```
example.go

type MailAdapter struct {
    client ExternalMailClient
}

func (m MailAdapter) Send(to string, subject string, body string) error {
    payload := MailPayload{
        To:      to,
        Subject: subject,
        Body:    body,
    }

    return m.client.SendEmail(payload)
}
```



Patrón middleware

Patrón middleware: Permite ejecutar lógica antes y después de una petición HTTP.

```
example.go

func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        slog.Info("request", "method", r.Method, "path", r.URL.Path)
        next.ServeHTTP(w, r)
    })
}
```



Inyección de dependencias

Algunos no lo consideran un patrón pero es importante en Go. Sirve para pasar dependencias desde afuera en lugar de crearlas dentro del componente.

```
example.go

func NewTaskService() *TaskService {
    db := connectDB()
    repo := NewSQLTaskRepository(db)

    return &TaskService{repo: repo}
}
```

```
example.go

func NewTaskService(repo TaskRepository) *TaskService {
    return &TaskService{repo: repo}
}

service := NewTaskService(fakeRepo)
```



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

